

Customer Q&A Briefing GenAI Adoption and the Agentic Repository

Anticipated questions from Engineering and Imaging leadership,
with model answers for the account review



How to use this guide

The deck and workbook show the evidence. This guide prepares you for the questions that follow. It is organised by theme, and each item gives the likely asker, what is behind the question, a model answer, and where useful, a follow-up line or a phrase to avoid.

Principles for every answer

#	Principle	In practice
1	Controls before benefits.	Lead with how risk is controlled, then the value. Engineering and imaging leaders trust controls more than promises.
2	Label what you know.	Say Measured, Estimated or Survey every time. Never present sample or estimated figures as results.
3	Concede real limits quickly.	Review cannot prove absence of defects. Evidence is from two projects. Saying so first protects credibility.
4	Never guarantee.	No guarantee of zero defects, regulatory outcomes or savings. Offer traceability, conservative controls and measurement.
5	Defer, then deliver.	If you do not know (vendor terms, regulatory classification), say you will confirm in writing, and do.
6	Offer a boundary, do not argue a principle.	If someone objects to AI near clinical code, offer to exclude those modules rather than debate.

Phrases that work, and phrases to avoid

Say	Avoid
That is a fair challenge. Here is how we handle it.	It is very accurate.
We measure that and report it; here is the figure and its basis.	Trust us, it works.
We will confirm in writing with your security and quality teams.	That is not a concern.
It is an observation, not proof of cause.	The AI reviewed it, so it is fine.
You decide which modules are in scope.	It will never make a mistake.
We propose to agree stop criteria with you.	We will save you X percent.

Facts to have ready

These come from the reporting workbook. The values below are the sample figures used in the deck.

Refresh every figure from your real workbook before the meeting and do not quote a number you cannot trace to a sheet.

Fact	Sample value	Workbook source	Basis
Projects reporting	2 of 10	Dashboard	Count
Licences actively used	70% (16 of 23)	Metrics	Measured / Estimated
Merged PRs with AI-assisted review	33% (62 of 190)	Metrics	Measured / Estimated
Median change, review rounds per PR	-8.6%	Metrics	1 measured, 1 estimated
Median change, PR cycle time	-12.0%	Metrics	1 measured, 1 estimated
Developer survey	19 respondents; 68% report saving time	Metrics	Self-reported
AI findings accepted by humans	64% (50 of 78)	Agentic Repo	Shadow-mode log
False positives reported	10	Agentic Repo	Shadow-mode log
Approved rules / mined rules	12 / 4	Agentic Repo	Count
Highest-rated risks	Missed defects; traceability	Risks	Likelihood x impact

Questions to ask the customer

Taking the initiative lowers the temperature. Useful questions: Who owns the regulatory view of AI-assisted development tools? Which modules would you exclude from AI-assisted code generation? What evidence would convince you to scale beyond the pilot? Which quality indicators do you trust most for this product? Who should be our data owner in each project?

A. Patient safety, regulation and traceability

A1 If AI-assisted code causes a defect that reaches a hospital, who is accountable?

Likely from: Engineering + Imaging **Behind the question:** Testing whether AI dilutes ownership.

MODEL ANSWER

Accountability does not change. The engineer who submits a change owns it, and the architect who approves it signs off on it. AI is a tool in the process, not an author of record. Every change still goes through the same review, testing and release gates.

What the framework adds is more evidence, not less: each pull request carries a review report, rule citations and an approval trail. For clinical-critical code, a named architect sign-off is required regardless of what any tool reported.

If pressed: We would rather document this explicitly with your quality team than leave it implied.

Avoid: Do not say 'the tool reviewed it' as a reason a change was acceptable.

A2 Does this affect our IEC 62304, ISO 13485 or regulatory submissions?

Likely from: Engineering + Imaging **Behind the question:** Checking whether we will create regulatory work for them.

MODEL ANSWER

The product's software lifecycle requirements do not change, and this approach sits inside the existing process. It does not replace any planning, verification or release activity.

What does need a decision is how AI-assisted development is documented and how the tooling is treated in your quality system. That is a conversation for your quality and regulatory team, and we propose to have it early and agree the documentation approach jointly. We are not claiming any certification or compliance status for the framework itself.

Avoid: Never state or imply that the framework 'is compliant' or 'meets 62304'.

A3 Is Copilot treated as SOUP? Do we have to validate it?

Likely from: Engineering **Behind the question:** Probing our understanding of tool qualification.

MODEL ANSWER

That classification is your regulatory team's decision, and we would not pre-empt it. Our design principle is that no tool output is trusted on its own: every change is verified by independent means, namely human review, automated tests and static analysis. That is the usual basis for proportionate confidence in a development tool.

We suggest a short, risk-based assessment of the tool and its intended use, recorded in your quality system, and we are glad to draft the technical input.

Avoid: Avoid giving a definitive regulatory classification.

A4

Can the agent touch rendering, window/level, measurements, orientation or patient-matching code?

Likely from: Imaging **Behind the question:** The core fear: silent change in what a clinician sees.

MODEL ANSWER

You decide that. The framework has a list of critical paths, which you define, and any change touching those files is flagged automatically and requires architect sign-off and test evidence. The agent can propose changes but cannot merge them.

If you prefer, we can go further and exclude specific modules from AI-assisted code generation altogether while still using AI for review comments, documentation and tests.

Avoid: Do not promise the AI 'will never' produce a wrong change; promise the controls.

A5

How do you know AI did not subtly change DICOM behaviour: tags, value representations, transfer syntax, pixel data handling?

Likely from: Imaging **Behind the question:** Imaging head testing technical depth and the safety net.

MODEL ANSWER

Two layers. First, a rule requires that wire formats and public contracts (DICOM, HL7, FHIR, WADO-RS, REST) are not changed without a recorded decision and a compatibility check, and the reviewer flags such areas. Second, and more important, your existing regression and conformance tests remain the safety net.

To be direct: review, human or AI, cannot prove the absence of a behavioural change. Tests can detect it, which is why a bug fix or contract-adjacent change must carry a test or a stated reason why not.

A6

Can we trace which changes were AI-assisted, for audit?

Likely from: Engineering **Behind the question:** Auditability.

MODEL ANSWER

Yes, to the level you want. Review reports are generated per pull request, rules carry approver names and dates, and everything sits in version control. We can add a lightweight convention, such as a pull request template field or commit trailer, to mark AI-assisted changes. We would agree the convention with your quality team.

B. Quality and reliability of the AI

B1 AI hallucinates. Why should I trust any of its findings?

Likely from: Engineering + Imaging **Behind the question:** The most common challenge.

MODEL ANSWER

You should not trust it blindly, and the design assumes that. A finding must cite a rule and quote the exact code. A script then checks that the quoted code really exists at that location, and findings that fail are excluded from the score. Deterministic checks, such as the language-version ceiling, are plain scripts with unit tests rather than AI judgement.

Then humans decide. We also measure quality: the share of findings accepted by reviewers and the number of false positives reported. Where acceptance is low, we fix the rule, not the people.

If pressed: Sample figure: 64% acceptance (50 of 78) in the pilot. Refresh from the workbook before quoting.

Avoid: Avoid 'it is very accurate' without a number and a basis.

B2 What are the false-positive and miss rates?

Likely from: Engineering **Behind the question:** Wants numbers, and will notice evasion.

MODEL ANSWER

False positives we measure directly through accepted versus rejected findings, and we report them. Misses are harder: you cannot count what nobody found. We use proxies such as escaped defects and rework, compared with the baseline period.

That is why AI review is an additional layer and never a replacement for human review, and why critical paths keep full human scrutiny.

Avoid: Do not claim a miss rate you have not measured.

B3 We already run SonarQube and other analysers. What does this add?

Likely from: Engineering **Behind the question:** Overlap and tool sprawl.

MODEL ANSWER

Those tools stay in place and remain authoritative for what they cover. This adds what generic analysers cannot know: your architecture constraints, your agreed conventions, and context from the pull request. The reviewer is instructed not to duplicate analyser findings.

B4 The model behind Copilot changes without notice. How do you keep results stable?

Likely from: Engineering **Behind the question:** Reproducibility concern.

MODEL ANSWER

The deterministic parts do not change. The AI-judgement part can, so we monitor it: acceptance and false-positive rates are tracked, and if they move after a model change, we re-run the shadow-mode comparison before relying on it. The rules themselves are plain text that you own, not behaviour locked inside a model.

B5**Our product is on .NET Framework 4.8 and C# 7.3. Models love modern syntax and APIs.**

Likely from: Engineering **Behind the question:** Knows the typical failure.

MODEL ANSWER

Agreed, that is a known failure mode. Language level is covered by an automated check for syntax newer than C# 7.3, with unit tests, plus an instruction. API availability is harder to check by script, so the reviewer must verify against the project references and mark anything uncertain as 'verify'. The compiler catches the remainder at build time.

B6**Will developers over-rely on it and our review quality drop?**

Likely from: Engineering **Behind the question:** Skills erosion and review fatigue.

MODEL ANSWER

It is a real risk and we manage it rather than deny it. The author owns and must be able to explain every line. Reviews cite rules, so reviewers see why. We monitor review comment categories and escaped defects, and if quality indicators worsen, we change the practice. Adoption without measured quality is not a success for us.

Avoid: Do not dismiss it as 'not a concern'.

B7**Will AI produce bigger pull requests and more review load?**

Likely from: Engineering **Behind the question:** Throughput concern.

MODEL ANSWER

The implementer is instructed to make the smallest change that meets the requirement and avoid speculative abstraction. We track pull request size and cycle time, and if either rises, that is a stop signal for us.

C. Data, IP and privacy

C1

Does our source code or patient data leave our boundary, or train anyone's models?

Likely from: Engineering + Imaging **Behind the question:** Security gatekeeper question.

MODEL ANSWER

Patient data should never be near these tools, and the framework has a rule for it: synthetic data only in code, tests, prompts and logs. We recommend technical controls to back it up, such as secret scanning and no production data on development machines.

On source code, the answer depends on the licence tier and the vendor's data-handling terms. We will confirm the exact terms in writing with your security team rather than quote them from memory, and the framework works within whatever boundary you set.

Avoid: Do not quote vendor data-retention terms from memory.

C2

Who owns the rules and the learned knowledge?

Likely from: Engineering **Behind the question:** IP and lock-in.

MODEL ANSWER

The rules derive from your standards and your reviewers' judgement, so they should live in your repository under your ownership. Contractual detail is for the commercial discussion, but our intent is clear: plain text files and standard scripts, nothing proprietary.

C3

Review comments contain names and confidential details. Is mining them a privacy problem?

Likely from: Engineering **Behind the question:** Internal sensitivity.

MODEL ANSWER

Mining runs inside your organisation. Exports stay out of version control, pipeline artifacts expire after 14 days, and rules are generalised: they carry no author names, ticket content or patient data. An architect reads every candidate before it becomes a rule.

C4

Can we see and audit exactly what the agents are told to do?

Likely from: Engineering **Behind the question:** Transparency.

MODEL ANSWER

Yes. Instructions, agent definitions, prompts and rules are plain text in the repository, version controlled and changed only through pull requests. You can read all of it, and we recommend your architects do.

D. Evidence and metrics

D1 These numbers come from two projects out of ten. Why should we believe them?

Likely from: Engineering + Imaging Behind the question: Evidence quality.

MODEL ANSWER

You should treat them as pilot-level evidence, and that is how we present them. Eight projects are not yet reporting and we do not extrapolate. Every figure is labelled Measured, Estimated or Survey, and we report them separately.

The proposal is to baseline all ten projects before drawing account-wide conclusions.

Avoid: Do not generalise to the account.

D2 How do you know it is the AI that improved things?

Likely from: Engineering Behind the question: Causality.

MODEL ANSWER

We do not claim causality; we report observations. We use equal-length baseline and current periods and report alongside other indicators. A staggered rollout across projects also gives a natural comparison, with projects that have not started acting as reference points.

D3 Fewer review rounds could just mean less scrutiny. What about quality?

Likely from: Imaging Behind the question: Excellent challenge.

MODEL ANSWER

Fewer rounds is only good if quality holds, so we add counter-measures: escaped defects, rework, and review comment categories, captured in the baseline. Any increase in escaped defects, particularly in critical paths, is investigated before anything else.

D4 Self-reported time savings are unreliable.

Likely from: Engineering Behind the question: Credibility of survey data.

MODEL ANSWER

Agreed. That is why survey data is labelled as such and is not used in our headline figures. The headline metrics come from pull request data.

D5 Why did Project B improve less, and why does it have no approved rules?

Likely from: Engineering Behind the question: Probing whether we hide weaker results.

MODEL ANSWER

Project B is at an earlier maturity level: a pilot, with partial instructions and starter rules still awaiting architect approval. The pattern is what we would expect: results follow maturity. We show it rather than average it away.

D6 What would make you stop or roll back?

Likely from: Engineering + Imaging Behind the question: Tests whether there is real discipline.

MODEL ANSWER

We would pause on any of these: rising escaped defects, acceptance below roughly 50% after rule tuning, a privacy incident, or growing reviewer load. We suggest agreeing these criteria with you in advance.

D7 What is the productivity gain in hours or money?

Likely from: Engineering **Behind the question:** Business case.

MODEL ANSWER

We do not present an unvalidated percentage. Our focus is on review cycles and rework, which we can measure from pull request data. Once the ten projects are baselined, we will report measured effects and clearly separate any estimates.

Avoid: Do not quote effort-reduction percentages that are estimates without saying so.

E. Engineering practicality

E1 Our code is legacy: .NET Framework, WPF, C++. Will AI help or add noise?

Likely from: Engineering Behind the question: Fit for their stack.

MODEL ANSWER

Instructions constrain the tool to your language level and patterns. We start where risk is lowest and value is clearest: review support, test drafts and documentation. Code generation is used where patterns are well established. C++ and the managed-native boundary get extra caution, with explicit ownership rules.

E2 Performance-critical paths, such as cine playback and large pixel data, can regress with naive code.

Likely from: Imaging Behind the question: Real imaging concern.

MODEL ANSWER

Agreed. Performance-critical modules should be treated as critical paths, with benchmarks or performance tests as part of acceptance. AI review on resource handling is advisory only. Specialist engineers remain the decision makers there.

E3 How does it cope with Angular 21 alongside older prototype-based JavaScript?

Likely from: Engineering Behind the question: Mixed codebase.

MODEL ANSWER

Path-specific instructions apply different guidance to different file types. Legacy files keep their existing style, and nothing is converted to a new style unless someone asks.

E4 What does it cost in effort to set up and maintain?

Likely from: Engineering Behind the question: Hidden cost.

MODEL ANSWER

The main set-up effort is architects completing the design-constraints document, which is valuable regardless of AI. After that, we estimate roughly one to two hours a month of architect time for rule curation; we will confirm from actual experience. Licence costs depend on your current agreements and we do not assume them here.

E5 With ten projects, won't rules conflict or go stale?

Likely from: Engineering Behind the question: Governance at scale.

MODEL ANSWER

We separate shared core rules from project-specific design constraints. Each rule has an owner approval and a review-by date, and rules that do not earn their place are retired.

E6 Won't our architects become the bottleneck approving rules?

Likely from: Engineering Behind the question: Capacity.

MODEL ANSWER

Approval is monthly and limited to patterns that recur across several pull requests, which are the comments architects are already writing repeatedly. The aim is to write them once.

F. Commercial and relationship

F1 Will you pass the savings to us or bill the same?

Likely from: Engineering Behind the question: Commercial fairness.

MODEL ANSWER

That is a fair question and a commercial one for the right forum. Our proposal is to measure first, then agree how to treat benefits based on measured, not estimated, results. Transparency on the numbers comes before the discussion on terms.

Avoid: Do not commit to commercial terms in a technical review.

F2 Are you planning to reduce team size?

Likely from: Engineering Behind the question: Anxiety about people.

MODEL ANSWER

The goal is fewer review cycles and less rework, not reduced capacity. Any change in staffing would be a joint decision and not a by-product of this initiative.

Avoid: Do not speculate about staffing.

F3 What if the vendor changes pricing or features? Are we locked in?

Likely from: Engineering Behind the question: Lock-in.

MODEL ANSWER

Rules, scripts and design constraints are plain text and tool-agnostic. Only the naming of instruction files is specific to the current tool, so the investment is portable.

F4 Why this tool and not another?

Likely from: Engineering Behind the question: Vendor choice.

MODEL ANSWER

We are using what is already approved and available in your environment. The approach is portable, and any alternative can be assessed on the same metrics.

G. Difficult and sceptical questions

G1 Isn't this AI hype in a nicer wrapper?

Likely from: Engineering + Imaging **Behind the question:** Scepticism.

MODEL ANSWER

Fair to ask. Most of the framework is ordinary engineering: scripts, rules, approvals and metrics. AI is used narrowly where judgement is needed and is wrapped in checks. We have stop criteria, and we label estimates as estimates.

G2 Has AI-assisted work ever caused a problem?

Likely from: Engineering + Imaging **Behind the question:** Honesty test.

MODEL ANSWER

Answer only with facts you can document. If there have been issues, name them, say what was learned and what control changed. If not, say that none has been attributed to AI-assisted changes in the reporting period, and that false positives in shadow mode were [number] and are tracked.

If pressed: Confirm with the project leads before the meeting.

Avoid: Never say 'no problems' unless you have verified it.

G3 Your agent might codify our bad habits.

Likely from: Engineering **Behind the question:** Quality of learned rules.

MODEL ANSWER

That is why rules are not automatic. A rule needs evidence across several pull requests, no contradiction between reviewers, and an architect's approval. Conflicting views go to the architect as a decision. Rules expire for review, and rejected candidates are logged with reasons.

G4 Can you guarantee no regulatory finding?

Likely from: Imaging **Behind the question:** Impossible promise.

MODEL ANSWER

No one can guarantee that. What we can do is keep traceability, use conservative controls and involve your quality and regulatory team early, so that decisions are documented and defensible.

Avoid: Never guarantee outcomes.

G5 If our auditors ask us to switch it off in six months, what happens?

Likely from: Engineering **Behind the question:** Reversibility.

MODEL ANSWER

It is reversible. The framework is files and scripts, with no runtime dependency in the product. If switched off, existing code remains ordinary reviewed code, and we revert to the previous process.

G6

I do not want AI anywhere near a product used in hospitals.

Likely from: Imaging **Behind the question:** Principled objection.

MODEL ANSWER

We would distinguish AI in the product from AI in the development process. This proposal adds no AI features to the product and changes no clinical functionality. If you want a stricter boundary, we can exclude critical modules from AI-assisted code generation and limit use to review comments, documentation and tests, with the same human approvals.

Avoid: Do not argue with the principle; offer a boundary.